

กลุ่ม 12 — References & Borrowing

1. Mr.UDTARAKVISETH LAY — Concept + Short Code Illustration
2. นายกรันต์ชัย คำทรัพย์ — Detailed Code + Live Demo
3. นางสาวณัฐณิชา ภู่วงษ์ — Rust vs Other Language + PPL Analysis
4. นายเทพพิทักษ์ นิลดำ — Exercises + Common Mistakes + Challenge

Agenda — โครงสร้างการนำเสนอ (รวม ~20 นาที)

แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ

1

Member 1

**Concept + Short Code
Illustration**

แนวคิดหลักของหัวข้อ พร้อมโค้ดตัวอย่างสั้น ๆ

~5 นาที

2

Member 3

**Rust vs Other Language +
PPL Analysis**

เปรียบเทียบภาษา และวิเคราะห์เชิง
Programming Language

~5 นาที

3

Member 4

**Exercises + Common
Mistakes + Challenge**

ข้อผิดพลาดที่พบบ่อย และโจทย์ท้าทายผู้ฟัง

~5 นาที

Immutable References (การอ้างอิงแบบอ่านอย่างเดียว)

- **Borrowing (การยืม):** การใช้ Reference (&) เพื่อเข้าถึงข้อมูล โดยไม่รับช่วงความเป็นเจ้าของ (Ownership).
- **อ่านอย่างเดียว:** &T เป็นการยืมเพื่ออ่านเท่านั้น ไม่สามารถแก้ไขข้อมูล ต้นฉบับผ่าน Reference นี้ได้
- **การคืนสิทธิ์:** เมื่อฟังก์ชันทำงานจบ Ownership ยังคงอยู่ที่ตัวแปรต้นฉบับ และ สามารถใช้งานต่อได้ทันที

```
fn main() {
    let my_string: String = String::from("Silpakorn");

    // ส่ง &my_string เป็น Immutable Reference
    let length: usize = calculate_length(&my_string);

    // my_string ยังคงใช้งานได้ตามปกติ
    println!("The length of '{}' is {}.", my_string, length);
}

fn calculate_length(s: &String) -> usize {
    s.len()
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
The length of 'Silpakorn' is 9.
```

Mutable References (การอ้างอิงแบบแก้ไขค่าได้)

- **ยืมเพื่อแก้ไข:** การใช้ `&mut T` อนุญาตให้ฟังก์ชันหรือผู้ยืมสามารถเปลี่ยนแปลงข้อมูลต้นฉบับได้
- **เงื่อนไขสำคัญ:** ตัวแปรเจ้าของข้อมูลต้องถูกประกาศด้วยคีย์เวิร์ด `mut` ก่อนถึงจะสร้าง Mutable Reference ได้
- **ส่งต่อสิทธิ์:** ต้องใส่ `&mut` ทั้งตอนที่ส่งค่าเข้าฟังก์ชัน และตอนที่ประกาศรับพารามิเตอร์ในฟังก์ชัน

```
fn main() {
    // ต้องประกาศตัวแปรต้นฉบับเป็น
    let mut greeting: String = String::from("Hello");
    // ส่ง &mut greeting เข้าไปแก้ไข
    append_text(&mut greeting);
    println!("After borrowing mutably: {}", greeting);
}

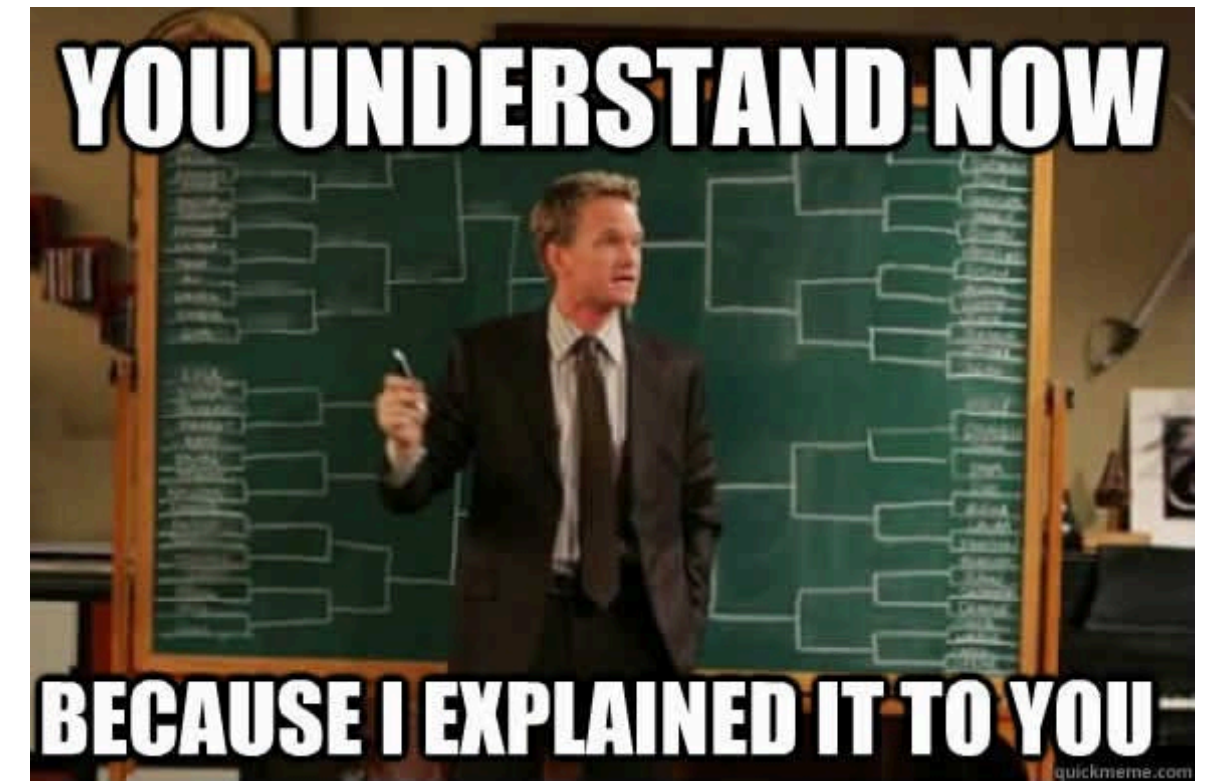
fn append_text(s: &mut String) {
    s.push_str(string: ", Rust!");
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
After borrowing mutably: Hello, Rust!
```

Syntax และกฎของ Borrow Checker

- Syntax พื้นฐาน:
 - **&T** : ยืมค่าเพื่ออ่านอย่างเดียว (let ref_val = &my_string;)
 - **&mut T** : ยืมค่าเพื่ออ่านและแก้ไข (let mut_ref = &mut my_string;)
- กฎข้อที่ 1 (เลือกได้อย่างเดียว): ในเวลาเดียวกัน สามารถมี **&mut** ได้เพียง 1 ตัว หรือ มี **&** ได้หลายตัว (ห้ามมีพร้อมกันเด็ดขาดเพื่อป้องกัน Data Race)
- กฎข้อที่ 2 (ความปลอดภัย): **Reference** ทุกตัวต้องชี้ไปยังข้อมูลที่ยังมีชีวิตอยู่เสมอ (Rust จะป้องกัน Dangling Reference ให้ตั้งแต่ตอนคอมไพล์)



Rust vs Python

Aspect	Rust	[Other Language]
Syntax	ใช้ & สำหรับ Reference, &mut สำหรับ Mutable Reference และ * สำหรับ Dereference	มี Reference โดยอัตโนมัติ แต่ไม่มี Syntax สำหรับ Borrowing โดยตรง
Semantics / Behavior	Compiler ตรวจสอบ Ownership และ Borrowing ตั้งแต่ Compile Time	Reference สามารถชี้ไปยัง Object เดียวกันได้ และ Memory ถูกจัดการอัตโนมัติ
Type System	Statically Typed มี Reference Type เช่น &T, &mut T ซึ่งกำหนดลักษณะการเข้าถึงและสิทธิ์ในการแก้ไขข้อมูล	Dynamically Typed ตัวแปรไม่จำเป็นต้องประกาศ Type และ Reference ถูกจัดการในระดับ Object
Memory Management	ใช้ Ownership และ Borrowing โดยไม่ต้องใช้ Garbage Collector เป็นกลไกหลัก	ใช้ Automatic Memory Management และ Garbage Collection
Safety	Compiler ช่วยตรวจสอบ Memory Safety และกฎการ Borrowing	จัดการ Memory อัตโนมัติ แต่ Type และข้อผิดพลาดหลายอย่างตรวจพบขณะ Runtime

Analysis

- ความแตกต่างสำคัญ: Rust แยกแนวคิด Ownership กับ Borrowing ออกจากกันอย่างชัดเจน โดย `&text` บอกว่า Function รับ Reference ไปยังข้อมูลแทนการรับ Ownership ขณะที่ Python ใช้ระบบ Object Reference โดยไม่มี Syntax ที่ผู้เขียนโปรแกรมต้องใช้เพื่อระบุ Borrowing แบบ Rust
- เหตุผลด้านการออกแบบ: Rust ถูกออกแบบให้สามารถตรวจสอบ Memory Safety ตั้งแต่ Compile Time โดยไม่ต้องพึ่ง Garbage Collector เป็นกลไกหลัก จึงต้องทำให้ความสัมพันธ์ระหว่างผู้เป็นเจ้าของข้อมูลกับผู้ที่ยืมข้อมูลมีความชัดเจน ส่วน Python เน้นความเรียบง่ายและความสะดวกในการเขียนโปรแกรม จึงจัดการ Memory ให้โดยอัตโนมัติ ทำให้ผู้เขียนโปรแกรมไม่จำเป็นต้องระบุการ Borrowing ใน Syntax

Rust vs Java

Aspect	Rust	[Other Language]
Syntax	ใช้ & สำหรับ Reference, &mut สำหรับ Mutable Reference และ * สำหรับ Dereference	ใช้ Reference ของ Object โดยตรง แต่ไม่มี Borrowing Syntax แบบ Rust
Semantics / Behavior	Compiler ตรวจสอบ Ownership และ Borrowing ตั้งแต่ Compile Time	Reference ใช้เข้าถึง Object และ Object ถูกจัดการโดย Garbage Collector
Type System	Statically Typed มี Reference Type เช่น &T, &mut T ซึ่งกำหนดลักษณะการเข้าถึงและสิทธิ์ในการแก้ไขข้อมูล	Statically Typed ต้องระบุ Type และ Reference จะระบุชนิดของ Object เช่น String หรือ Object
Memory Management	ใช้ Ownership และ Borrowing โดยไม่ต้องใช้ Garbage Collector เป็นกลไกหลัก	ใช้ Garbage Collector จัดการ Memory ของ Object โดยอัตโนมัติ
Safety	Compiler ช่วยตรวจสอบ Memory Safety และกฎการ Borrowing	มี Memory Safety จากการใช้ Reference และ Garbage Collector แต่ไม่มี Borrow Checker แบบ Rust

Analysis

- **ความแตกต่างสำคัญ:** *Rust* กำหนดสิทธิ์ในการเข้าถึงข้อมูลอย่างชัดเจนผ่าน `&` และ `&mut` และ *Compiler* ตรวจสอบว่าการ *Borrow* ไม่ขัดกับกฎของ *Ownership* ขณะที่ *Java* ใช้ *Reference* ที่สามารถอ้างถึง *Object* และใช้ *Garbage Collector* จัดการ *Memory* ของ *Object* ที่ไม่สามารถเข้าถึงได้แล้ว
- **เหตุผลด้านการออกแบบ:** *Rust* เลือกใช้ *Ownership* และ *Borrowing* เพื่อให้สามารถควบคุม *Memory* และตรวจสอบความปลอดภัยได้ตั้งแต่ *Compile Time* โดยไม่ต้องมี *Garbage Collector* ส่วน *Java* ถูกออกแบบให้เน้น ความง่ายในการพัฒนา และการจัดการ *Memory* อัตโนมัติ จึงใช้ *Garbage Collector* เพื่อจัดการ *Memory* ของ *Object* ที่ไม่สามารถเข้าถึงได้แล้ว ทำให้ *Programmer* ไม่ต้องจัดการ *Memory* ด้วยตนเอง

Rust vs C

Aspect	Rust	[Other Language]
Syntax	ใช้ & สำหรับ Reference, &mut สำหรับ Mutable Reference และ * สำหรับ Dereference	ใช้ Pointer * และ Address-of & ในการเข้าถึง Memory
Semantics / Behavior	Compiler ตรวจสอบ Ownership และ Borrowing ตั้งแต่ Compile Time	Pointer สามารถเข้าถึงข้อมูลผ่าน Address และแก้ไขข้อมูลใน ส่วนที่ Pointer ชี้ได้โดยตรงตามชนิดข้อมูลและการใช้งาน
Type System	Statically Typed มี Reference Type เช่น &T, &mut T ซึ่ง กำหนดลักษณะการเข้าถึงและสิทธิ์ในการแก้ไขข้อมูล	Statically Typed ต้องระบุ Type ของตัวแปรและ Pointer เช่น int *p
Memory Management	ใช้ Ownership และ Borrowing โดยไม่ต้องใช้ Garbage Collector เป็นกลไกหลัก	Programmer สามารถจัดการ Dynamic Memory โดยใช้ malloc() สำหรับจัดสรร Dynamic Memory และ free() สำหรับ คืน Memory ที่จัดสรรไว้
Safety	Compiler ช่วยตรวจสอบ Memory Safety และกฎการ Borrowing	มีความเสี่ยงจาก Pointer เช่น Dangling Pointer และ Memory Leak

Analysis

- **ความแตกต่างสำคัญ:** แม้ *Rust* และ *C* จะมี `&` และ `*` ที่ดูคล้ายกัน แต่แนวคิดเบื้องหลังแตกต่างกัน โดย *C* ใช้ *Pointer* และ *Address* เพื่อให้ *Programmer* ควบคุม *Memory* ได้โดยตรง ขณะที่ *Rust* ใช้ *Reference* ภายใต้กฎ *Ownership* และ *Borrowing*
- **เหตุผลด้านการออกแบบ:** *C* ถูกออกแบบโดยให้ความสำคัญกับการควบคุม *Hardware* และประสิทธิภาพระดับต่ำ จึงเปิดให้เข้าถึง *Memory* ได้อย่างอิสระ ส่วน *Rust* ต้องการรักษาความสามารถในการควบคุม *Memory* และ *Performance* แบบภาษา *System Programming* แต่เพิ่ม *Compile-time Safety* เข้ามา จึงออกแบบ *Ownership* และ *Borrowing* เพื่อให้ *Compiler* ตรวจสอบการใช้ *Reference* และลดปัญหา เช่น *Dangling Pointer* และการเข้าถึงข้อมูลที่ไม่ถูกต้อง

PPL Perspective — มุมมองเชิง Programming Language

Why Rust?

Rust ใช้ References & Borrowing เพื่อสร้างสมดุลระหว่าง Safety, Reliability และ Performance โดย Compiler สามารถตรวจสอบกฎของ Ownership และ Borrowing ตั้งแต่ Compile Time

- Safety: ช่วยป้องกันการใช้ Reference ที่ไม่ถูกต้อง เช่น Reference ที่ชี้ไปยังข้อมูลที่หมดอายุแล้ว
 - Reliability: กำหนดกฎการเข้าถึงข้อมูลอย่างชัดเจน เช่น การควบคุมการใช้ Mutable และ Immutable References
 - Performance: สามารถส่งข้อมูลผ่าน Reference ได้โดยไม่จำเป็นต้องคัดลอกข้อมูลหรือโอน Ownership
- ดังนั้น References & Borrowing จึงเป็นแนวคิดสำคัญที่ช่วยให้ Rust จัดการ Memory ได้อย่างปลอดภัยและมีประสิทธิภาพ โดยไม่ต้องใช้ Garbage Collector เป็นกลไกหลักของภาษา

PPL Perspective — มุมมองเชิง Programming Language

Syntax

References & Borrowing ในภาษา Rust เกี่ยวข้องกับ Syntax ผ่านรูปแบบการสร้างและประกาศ Reference โดยมี Syntax ที่สำคัญ ได้แก่

- `&T` สำหรับ Immutable Reference
- `&mut T` สำหรับ Mutable Reference ตาม Rust Reference รูปแบบทั่วไปของ Reference คือ `&` ตามด้วย `mut` ที่อาจมีหรือไม่มีก็ได้ และตามด้วย Type นอกจากนี้ Syntax ของ Reference ยังปรากฏใน Function Parameters และ Return Types เช่น `&String` ทำให้ลักษณะการเข้าถึงข้อมูลและข้อจำกัดของ Reference สามารถแสดงออกใน Source Code ได้อย่างชัดเจน

Semantics

Semantics คือการศึกษาความหมายและพฤติกรรมของคำสั่งหรือโครงสร้าง (Construct) โดยอธิบายว่าเมื่อโปรแกรมถูกประมวลผลแล้ว คำสั่งนั้นจะทำงานอย่างไรและให้ผลลัพธ์แบบใดใน References & Borrowing สิ่งสำคัญคือ `&` และ `&mut` ไม่ได้เป็นเพียงสัญลักษณ์ทาง Syntax แต่มีความหมายเกี่ยวกับวิธีการเข้าถึงข้อมูล

PPL Perspective — มุมมองเชิง Programming Language

Memory / Resource

References & Borrowing มีความสัมพันธ์โดยตรงกับ Ownership ซึ่งเป็นกลไกสำคัญในการจัดการ Memory ของ Rust โดย Borrowing ทำให้สามารถนำข้อมูลไปใช้งานผ่าน Reference ได้โดยไม่ต้องโอน Ownership ให้กับผู้ที่นำข้อมูลไปใช้

Type System

References & Borrowing เกี่ยวข้องกับ Type System เพราะ Reference ใน Rust มี Type ของตัวเอง เช่น `&T` และ `&mut T` โดย Type เหล่านี้กำหนดลักษณะการเข้าถึงข้อมูลว่าเป็นการอ่านอย่างเดียวหรือสามารถแก้ไขข้อมูลได้

Abstraction

References & Borrowing เชื่อมโยงกับแนวคิด PPL อื่น ๆ เช่น Abstraction, Scope และ Ownership โดยเฉพาะ Abstraction ที่ช่วยให้ Function สามารถกำหนดเพียงว่าต้องการ “เข้าถึงข้อมูล” โดยไม่จำเป็นต้องรับผิดชอบ Ownership ของข้อมูลนั้น ส่วน Scope จะกำหนดขอบเขตการใช้งานของ Reference

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

Mistake 1 — [Mutable and Immutable Reference Mixup]

✗ Incorrect

```
fn main() {
    let mut num: Vec<i32> = vec![1, 2, 3];
    let first_number: &i32 = &num[0];
    num.push(4); // mutable borrow ขณะที่ immutable ยังไม่จบ
    println!("{}", first_number);
}
```

✓ Correct

```
fn main() {
    let mut num: Vec<i32> = vec![1, 2, 3];
    num.push(4);
    let first_number: &i32 = &num[0];
    println!("{}", first_number);
}
```

ทำไมถึงผิด? (Why?)

Rust ไม่อนุญาตให้ immutable borrow และ mutable borrow ของข้อมูลเดียวกันเกิดขึ้นพร้อมกัน เพราะ mutable borrow สามารถแก้ไขข้อมูลได้ ในขณะที่ immutable borrow ต้องสามารถอ่านข้อมูลได้โดยไม่ต้องกังวลว่าข้อมูลจะถูกแก้ไขระหว่างที่ borrow ยังใช้งานอยู่

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

Mistake 2 — [Multiple Mutable References]

✗ Incorrect

```
fn main() {
    let mut num: i32 = 10;

    let ref1: &mut i32 = &mut num;
    let ref2: &mut i32 = &mut num;
    // multiple mutable reference on top of each other
    *ref1 += 5;
    *ref2 += 10;

    println!("{}", num);
}
```

✓ Correct

```
fn main() {
    let mut num: i32 = 10;

    let ref1: &mut i32 = &mut num;
    *ref1 += 5;
    // makes another reference after the first one finishes
    let ref2: &mut i32 = &mut num;
    *ref2 += 10;

    println!("{}", num);
}
```

ทำไมถึงผิด? (Why?)

Rust มีกฎเกี่ยวกับการ borrow ว่า ในช่วงเวลาเดียวกัน トラบที่ borrow ยังถูกใช้งานอยู่ เราสามารถมี mutable borrow ได้เพียงหนึ่งตัวเท่านั้น เหตุผลที่ Rust ห้ามมี Mutable reference หลายตัวที่อ้างอิงข้อมูลเดียวกันพร้อมกัน เพราะ Mutable reference สามารถแก้ไขข้อมูลที่อ้างอิงได้ Rust จึงต้องให้ Mutable borrow สามารถเข้าถึงข้อมูลแบบ exclusive เพื่อป้องกันการเข้าถึงหรือแก้ไขข้อมูลเดียวกันจากหลายจุดในเวลาเดียวกัน

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

กลุ่มเพื่อนกลุ่มหนึ่งกำลังจะไปเที่ยวด้วยกัน โดย Noah ชวนเพื่อนมาได้อีก 2 คนคือ Oliver กับ James ในภายหลัง Quinn ตัดสินใจที่จะไปด้วย เหลือเพื่อนอีกคนชื่อ Charlotte ที่กำลังตัดสินใจว่าจะไปมัย โดยที่ตนคิดไว้ว่าจะไปถ้า Oliver ไปด้วย James จึงอยากเขียนโค้ดออกมาสั้นๆให้เห็นว่าใครเป็นคนชวน ไปเที่ยว รวมถึงคนที่จะไปด้วยตามลำดับเหตุการณ์ แต่โค้ดของเขากลับติด error

```
// James' code
fn main() {
    let mut names = vec![String::from("Noah"), String::from("Oliver"), String::from("James")];

    let first = &names[0];
    names.push(String::from("Quinn"));
    println!("Hosted by: {}", first);

    for name in &names {
        if name == "Oliver" {
            names.push(String::from("Charlotte"));
        }
    }
    println!("{:?}", names);
}
```

แต่โค้ดของเขากลับ compile ไม่ผ่าน จงหาว่า ทำไม compile ไม่ผ่านเพราะส่วนไหนบ้าง แล้วแก้ไขอย่างไรโดยการ restructure โค้ดดังกล่าว



Hint: ลองสังเกตการ borrow แต่ละอันว่าเริ่ม/จบตรงไหน

Solution

```
// Solution code
fn main() {
    let mut names = vec![String::from("Noah"), String::from("Oliver"), String::from("James")];

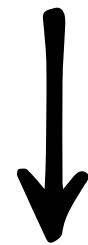
    let first = &names[0];
    println!("Hosted by: {}", first);
    names.push(String::from("Quinn"));

    let mut add = false;
    for name in &names {
        if name == "Oliver" {
            add = true;
        }
    }
    if add {
        names.push(String::from("Charlotte"));
    }
    println!("{:?}", names);
}
```

Solution



```
let first = &names[0];
names.push(String::from("Quinn"));
println!("Hosted by: {}", first);
```



```
let first = &names[0];
println!("Hosted by: {}", first);
names.push(String::from("Quinn"));
```

Explanation (คำอธิบาย)

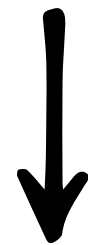
จุดที่ 1: first ถูกยืมใช้งานข้าม push() first เป็น immutable reference ที่ยังถูกใช้งานอยู่ ในขณะที่ names.push(...)

เรียกใช้งาน mutable borrow ของ names ซึ่ง Rust ไม่อนุญาต เนื่องจาก push อาจทำให้เกิด vector reallocate memory ทำให้ first กลายเป็น dangling reference ได้

วิธีแก้: ใช้ first ก่อน แล้วนำไปใช้งานเพื่อให้ borrow จบลงก่อนที่จะใช้ push

Solution

```
for name in &names {  
    if name == "Oliver" {  
        names.push(String::from("Charlotte"));  
    }  
}  
println!("{:?}", names);
```



```
let mut add = false;  
for name in &names {  
    if name == "Oliver" {  
        add = true;  
    }  
}  
if add {  
    names.push(String::from("Charlotte"));  
}  
println!("{:?}", names);
```

Explanation (คำอธิบาย)

จุดที่ 2: push ข้างในลูปที่กำลัง borrow names แบบ

immutable for name in &names ทำให้ names ถูกยืมแบบ

immutable ไปตลอดการวนลูป การจะ push เข้าไปใน names

ระหว่างนั้นต้องใช้ mutable borrow ซึ่งขัดกัน

วิธีแก้: แบ่งการทำงานเดิมออกเป็น 2 ส่วนคือ ส่วนการ

ตัดสินใจ และ ส่วนการเพิ่มข้อมูลเข้า

Key Takeaways — สรุปสิ่งที่ควรรจำ

1

References & Borrowing

- การใช้ References (&) ช่วยให้เราเข้าถึงข้อมูลได้โดยไม่ต้องย้ายความเป็นเจ้าของ ทำให้โค้ดส่วนต่างๆ สามารถอ่านข้อมูลเดียวกันได้อย่างปลอดภัย

2

Mutable vs. Immutable Borrows

- Rust บังคับกฎการยืมที่เข้มงวด: เราสามารถมี reference แบบอ่านอย่างเดียว (&T) ที่ตัวก็ได้ หรือ มี reference แบบแก้ไขได้ (&mut T) ได้เพียงตัวเดียว เพื่อป้องกัน Data Race

3

Compile-Time Aliasing Safety

- Rust ตรวจสอบอายุของ reference และกฎการยืมตั้งแต่ตอน Compile เพื่อความปลอดภัยของหน่วยความจำและป้องกันปัญหา Dangling References โดยไม่ต้องใช้ Garbage Collector

4

PPL Connection

- เชื่อมโยงกับแนวคิดหลักในวิชา PPL เช่น การควบคุมการอ้างอิงซ้ำซ้อน (Aliasing), Reference Semantics, และวิธีที่ภาษาโปรแกรมสมัยใหม่จัดการความปลอดภัยของพอยเตอร์

References — เอกสารอ้างอิง

1. [The Rust Programming Language — References and Borrowing](<https://doc.rust-lang.org/book/ch04-02-references-and-borrowing.html>)
2. [The Rust Programming Language — Ownership](<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>)
3. [Rust Reference — Borrow Expressions](<https://doc.rust-lang.org/reference/expressions/operator-expr.html#borrow-expressions>)
4. [Rust by Example — Borrowing](<https://doc.rust-lang.org/rust-by-example/scope/borrow.html>)

GitHub Repository: <https://github.com/Viseth101/rust-tutorial-2569>



That's the end

DONE WITH MY PRESENTATION



NOW I HAVE TO ANSWER QUESTIONS

Questions & Answers

กลุ่ม 12 — References & Borrowing